



Image-based Pattern Recognition Principles

CSE0613409

Machine Vision and Motion Analysis

Mahin Montasir Afif

(MMF)

Department of Computer Science and Engineering

Outline

- Introduction
- 2D Matched Filter
- Image Registration
- Bayes Statistical Classifier
- Neural Networks
- Syntactic Recognition
- Face Recognition

Introduction

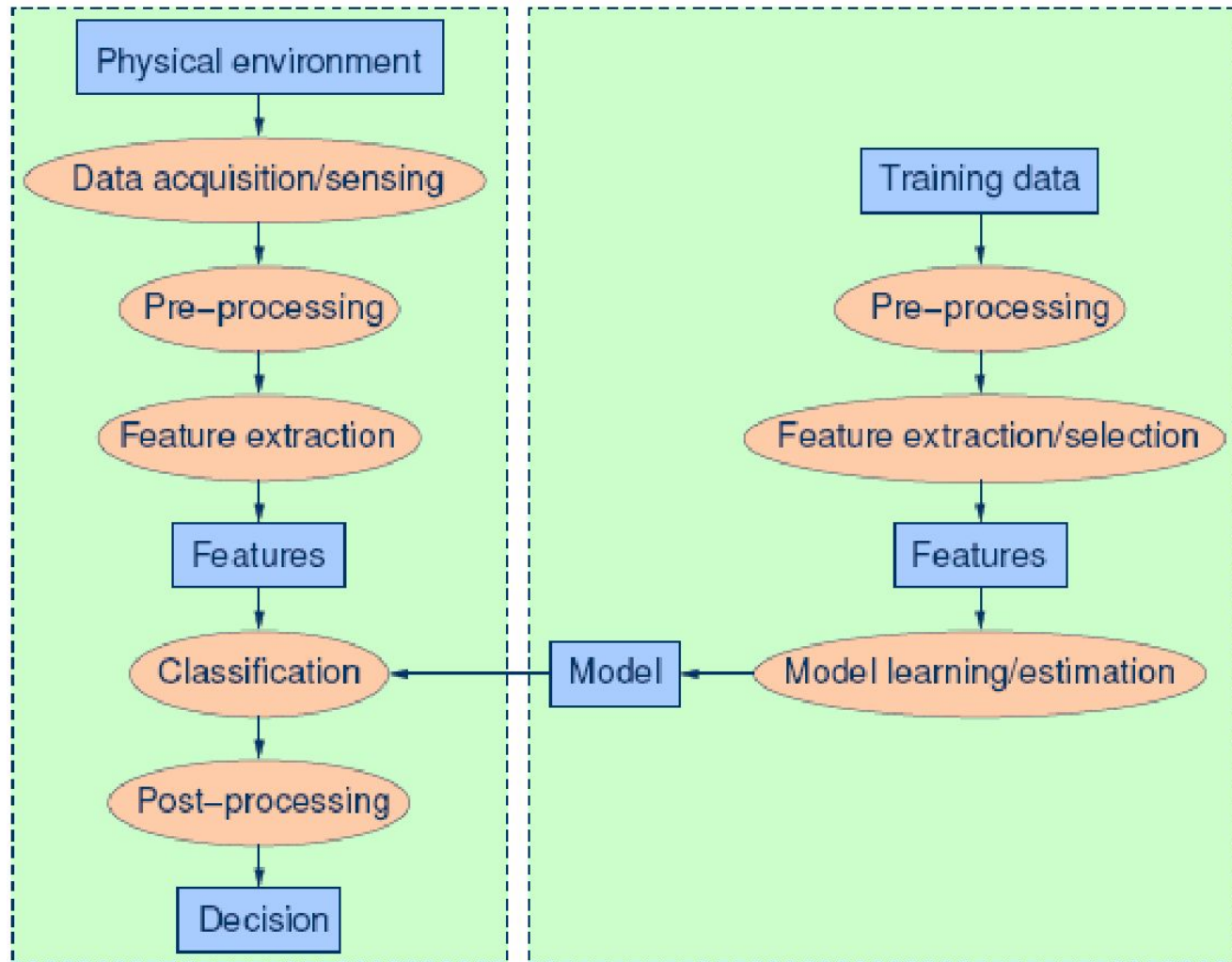


Fig.1 Basic components of a pattern recognition system[8]

Introduction

- **Data acquisition and sensing**
- **Pre-processing**
 - ◆ Removal of noise in data.
 - ◆ Isolation of patterns of interest from the background.
- **Feature extraction**
 - ◆ Finding a new representation in terms of features.
(Better for further processing)

Introduction

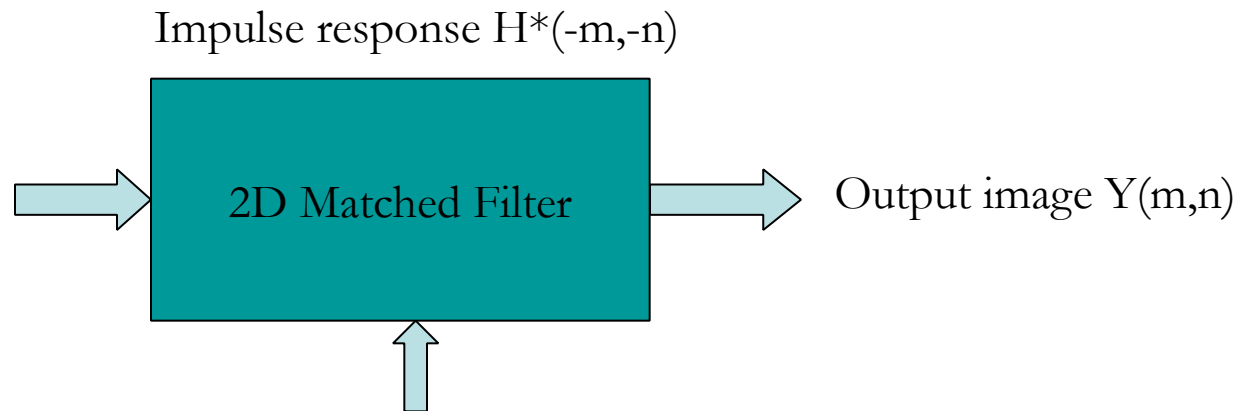
- **Model learning and estimation**
 - Learning a mapping between features and pattern groups.
- **Classification**
 - Using learned models to assign a pattern to a predefined category
- **Post-processing**
 - Evaluation of confidence in decisions.
 - Exploitation of context to improve performances.

2D Matched Filter

- **Functionality**
 - ◆ Degrading the noise effect.
 - ◆ Computing the similarity of two objects.
(Template matching for images)
- **Functional block**

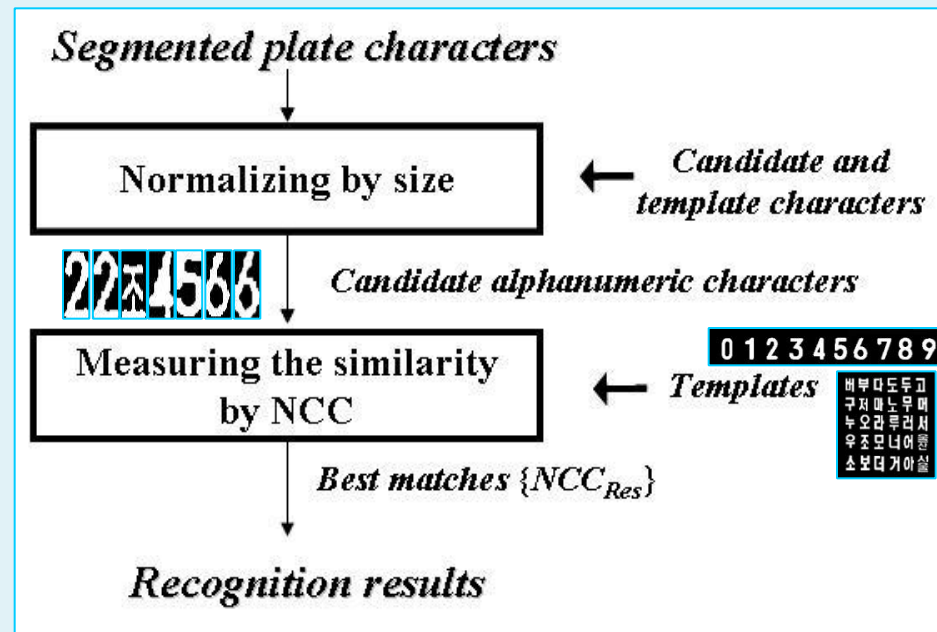


Input image $I(m,n)$



Template image $H^*(-m,-n)$

Character recognition



The proposed character recognition scheme



(a)



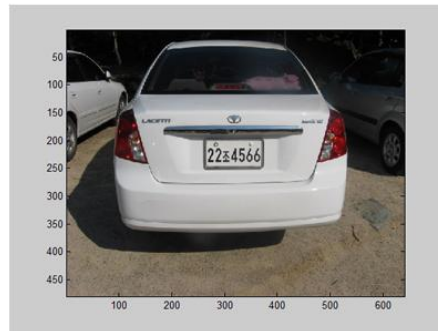
(b)

Template used for pattern matching: (a) 10 prototypes for the Korean plate numbers and (b) 30 prototypes for the Korean plate characters.

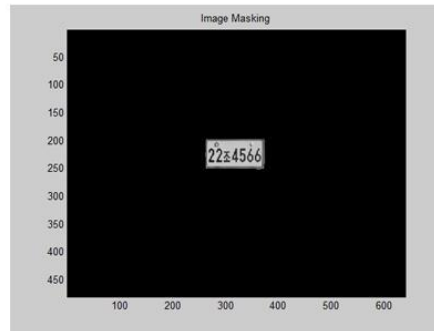
$$NCC_{ct} = \frac{\sum_{x=0}^{m-1} \sum_{y=0}^{n-1} (c - \bar{c}) (t - \bar{t})}{\sqrt{\sum_{x=0}^{m-1} \sum_{y=0}^{n-1} (c - \bar{c})^2 (t - \bar{t})^2}} \quad (5.1)$$

where NCC_{ct} is the correlation coefficient. The candidate plate character recognition process is based on the value of the correlation coefficient. If the value of the correlation coefficient exceeds a threshold set by the user, then the similarity measure is large enough and the input character can be assumed to present. Finally, a box on the target character

Processing example



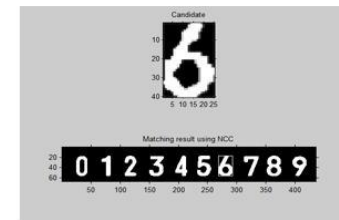
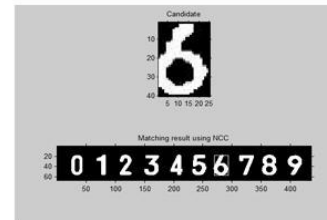
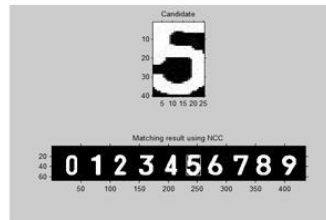
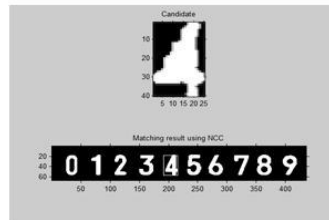
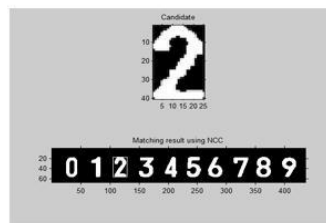
(a)



(b)



(c)



(d)

Illustration of LP segmentation and alphanumeric characters recognition: (a) an LP image, (b) extracted candidate region, (c) equal-sized candidate alphanumeric characters and (d) candidate alphanumeric characters measuring the similarity by NCC.

Processing example



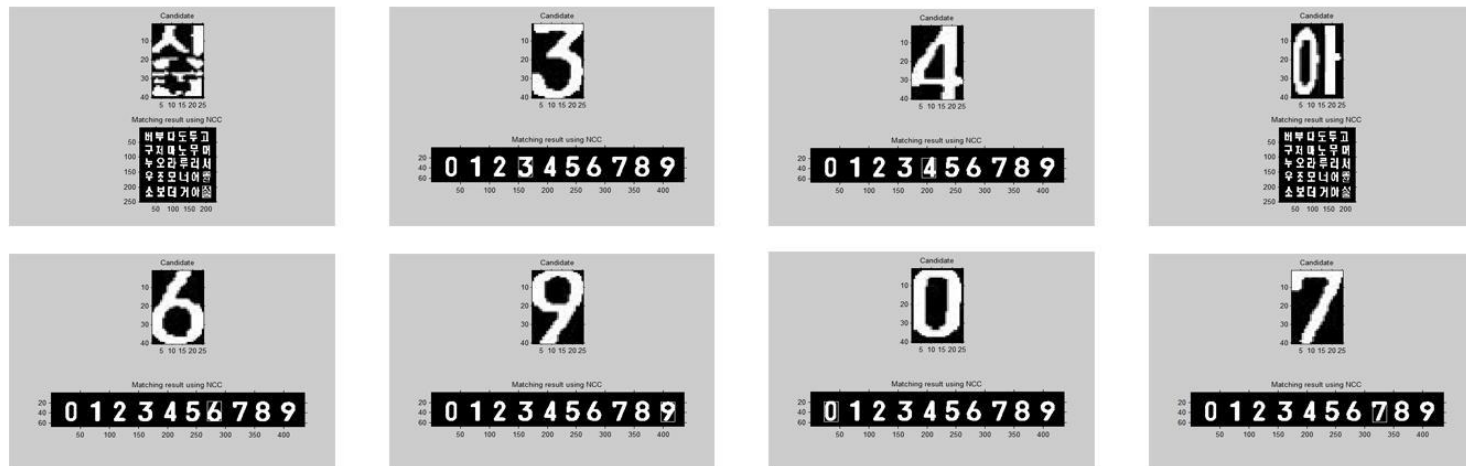
(a)



(b)



(c)



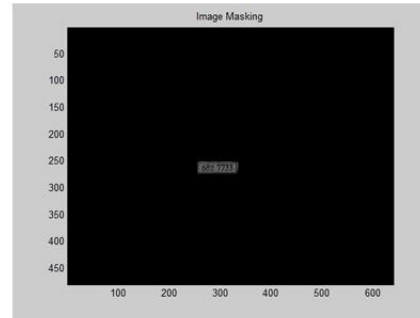
(d)

Illustration of LP segmentation and alphanumeric characters recognition: (a) an LP image, (b) extracted candidate region, (c) equal-sized candidate alphanumeric characters and (d) candidate alphanumeric characters measuring the similarity by NCC.

Processing example



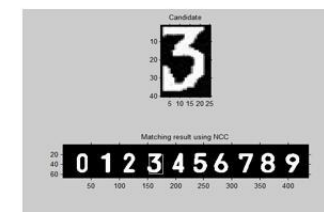
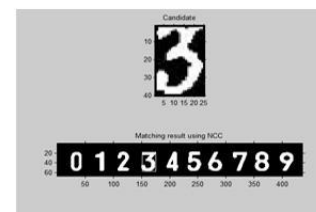
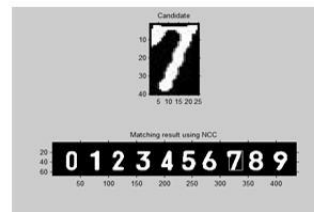
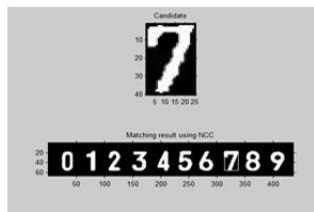
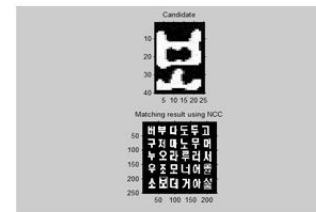
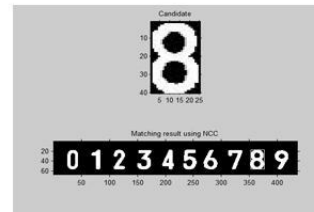
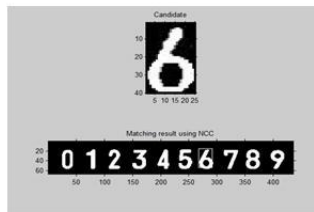
(a)



(b)



(c)



(d)

Illustration of LP segmentation and alphanumeric characters recognition: (a) an LP image, (b) extracted candidate region, (c) equal-sized candidate alphanumeric characters and (d) candidate alphanumeric characters measuring the similarity by NCC.

CHAIN CODE

- Two processes are included in this phase:
 - **Boundary extraction:** Image segmentation is performed by finding boundaries between objects
 - Steps are:
 - Mark potential edge points by finding discontinuities in features
 - Threshold the results
 - Merge edge segments into boundaries via edge linking
 - **Character segmentation**

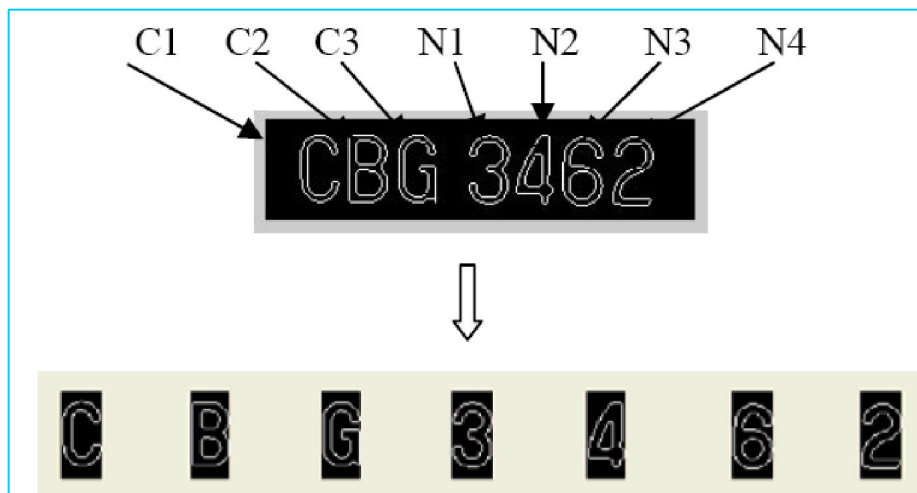


Fig. 4. Boundary image and segmented character regions

- The algorithm for extracting chain codes for 8-connected boundaries is as follows:
 - Find the pixel in the object that has the leftmost value in the topmost row
 - Define a variable $dir = 7$

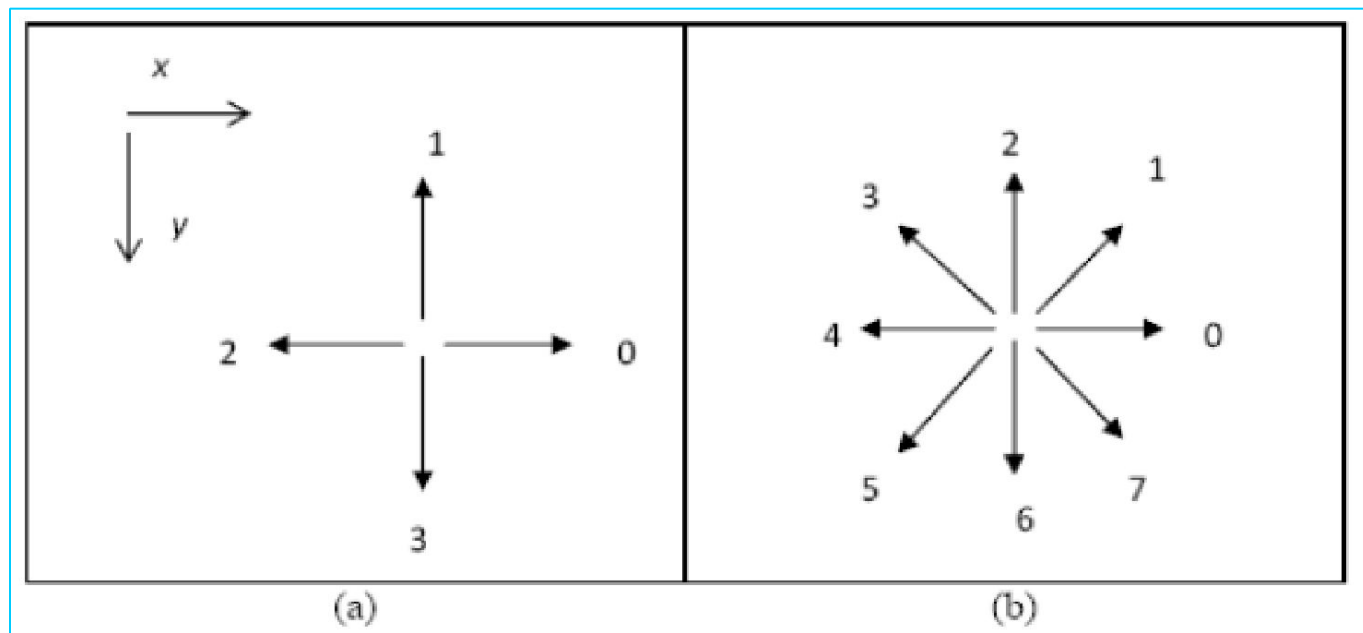


Fig. 5. Direction numbers for (a) 4-directional and (b) 8-directional chain code

- Traverse the 3x3 neighborhood of the current pixel in a counter-clockwise direction

- **Even**(0,2,4,6): $dir + 7 \% 8$
- **Odd** (1,3,5,7): $dir + 6 \% 8$

<i>dir</i>	0	1	2	3	4	5	6	7
$dir + 7 \pmod{8}$	7	0	1	2	3	4	5	6
$dir + 6 \pmod{8}$	6	7	0	1	2	3	4	5

- First foreground pixel will be new boundary element. Stop when the current boundary element P_n is equal to the second element P_1 and the previous boundary pixel P_{n-1} is equal to first boundary element P_0

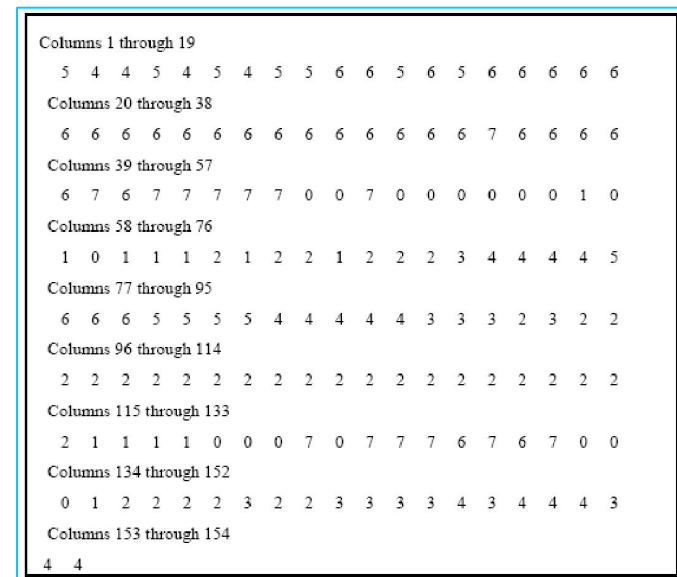
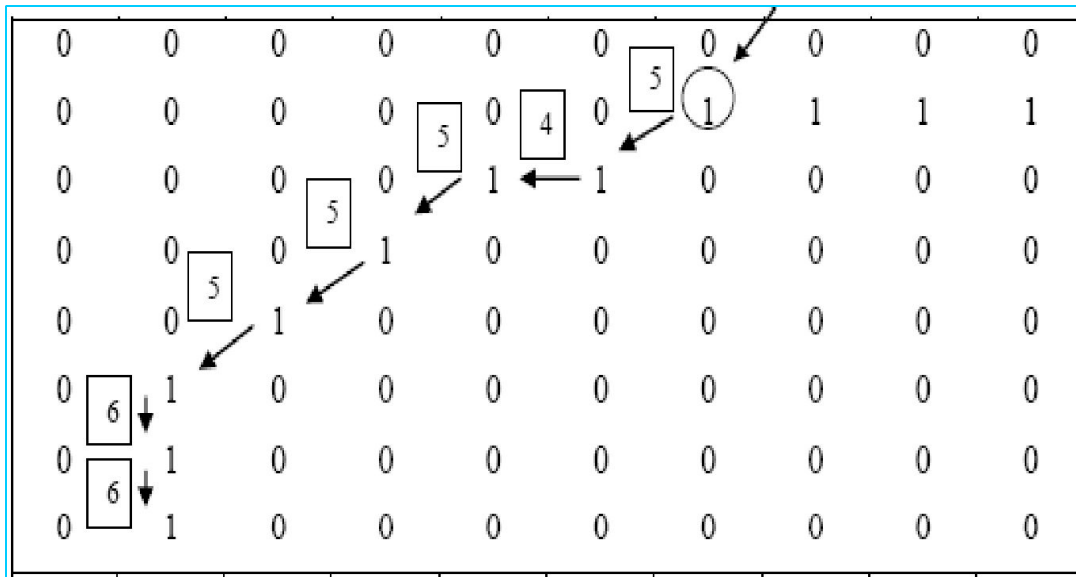
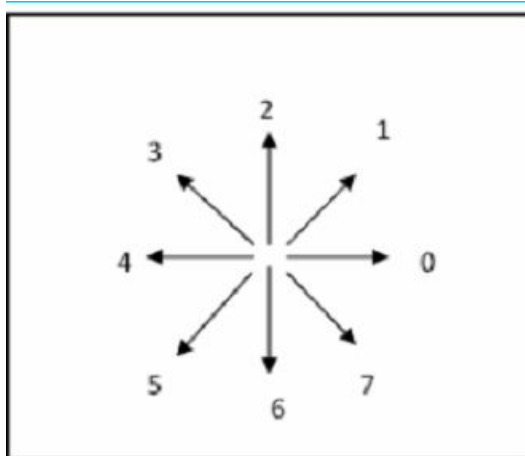
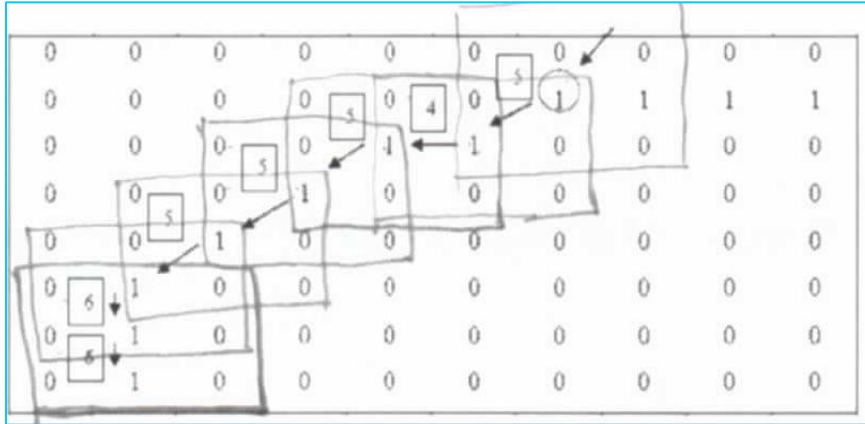


Fig. 7. The initial location and the direction to derive chain codes. And Fig. 8. the chain code extracted from the boundary image of character “C”



$dir = 7$

$(0, 2, 4, 6) \text{ even} : dir + 7 \% 8$

$(1, 3, 5, 7) \text{ odd} : dir + 6 \% 8$

$$\begin{aligned} 5 &\leftarrow 7 + 6 \% 8 = 5 \\ 4 &\leftarrow 5 + 6 \% 8 = 3 \\ 5 &\leftarrow 4 + 7 \% 8 = 3 \\ 5 &\leftarrow 5 + 6 \% 8 = 3 \\ 5 &\leftarrow 5 + 6 \% 8 = 3 \\ 6 &\leftarrow 3 + 6 \% 8 = 3 \\ 6 &\leftarrow 6 + 7 \% 8 = 5 \end{aligned}$$

different view same thing

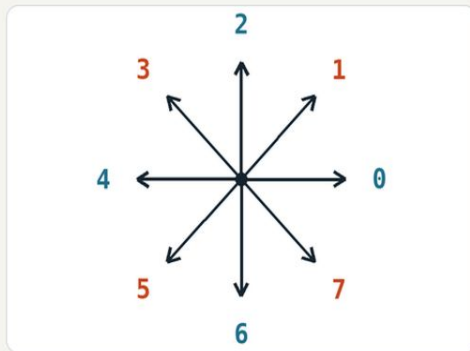


IMAGE PROCESSING · CONTOUR FOLLOWING

Boundary Tracing with 8-Direction Chain Code

Follow the edge of a shape one pixel at a time, turning the search direction by a parity rule.

DIRECTION WHEEL



Even = straight, Odd = diagonal.

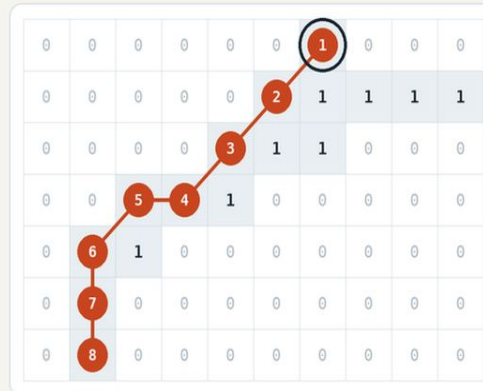
TURNING RULE

EVEN $\text{dir} = (\text{dir} + 7) \% 8$

ODD $\text{dir} = (\text{dir} + 6) \% 8$

Back the direction up so the next scan starts outside the shape: one slot after a straight step, two after a diagonal. Adding keeps it positive; % 8 wraps it.

TRACING ON THE PIXEL GRID



Red badges number the boundary pixels in visit order. The ringed pixel is the start, found by the top-left scan.

ALGORITHM

- 1 Scan top-to-bottom, left-to-right for the first 1.
- 2 Set the starting direction — here $\text{dir} = 7$.
- 3 Update dir with the parity rule.
- 4 Scan the 8 neighbours counter-clockwise from dir ; the first 1 is the next boundary pixel.
- 5 Record the code, move there, and repeat until you return to the start.

WORKED TRACE · $\text{dir} = 7$

STEP	COMPUTATION	DIR
1	$7 + 6 \% 8$	5
2	$5 + 6 \% 8$	3
3	$3 + 7 \% 8$	2
4	$2 + 7 \% 8$	1
5	$1 + 6 \% 8$	7
6	$7 + 7 \% 8$	6
7	$6 + 7 \% 8$	5

Each row applies the rule matching the parity of the current direction, then scans counter-clockwise from there.

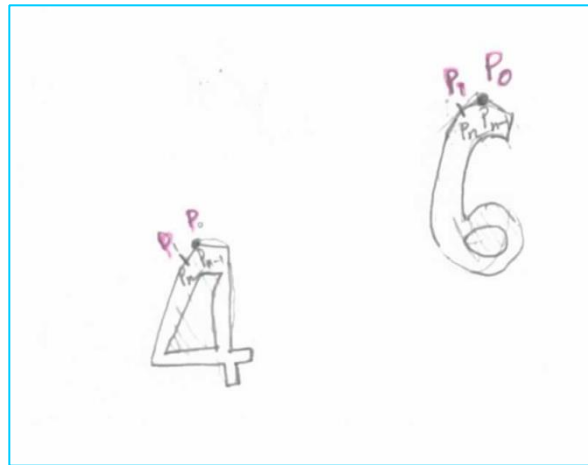
RESULT

chain code

The recorded direction codes form a compact description of the outline.

Binary image · 1 = object, 0 = background · 8-connectivity (Moore neighbourhood)

- **Character recognition has been done by using**
 - The list of chain codes derived for each character
 - Calculating the total number of each code direction contained in the list of chain code (*how many orders coordinates observe is important part in the rate of recognition*)
 - Total number of each code direction is used as a guide to recognize the characters



Eight-direction code vector

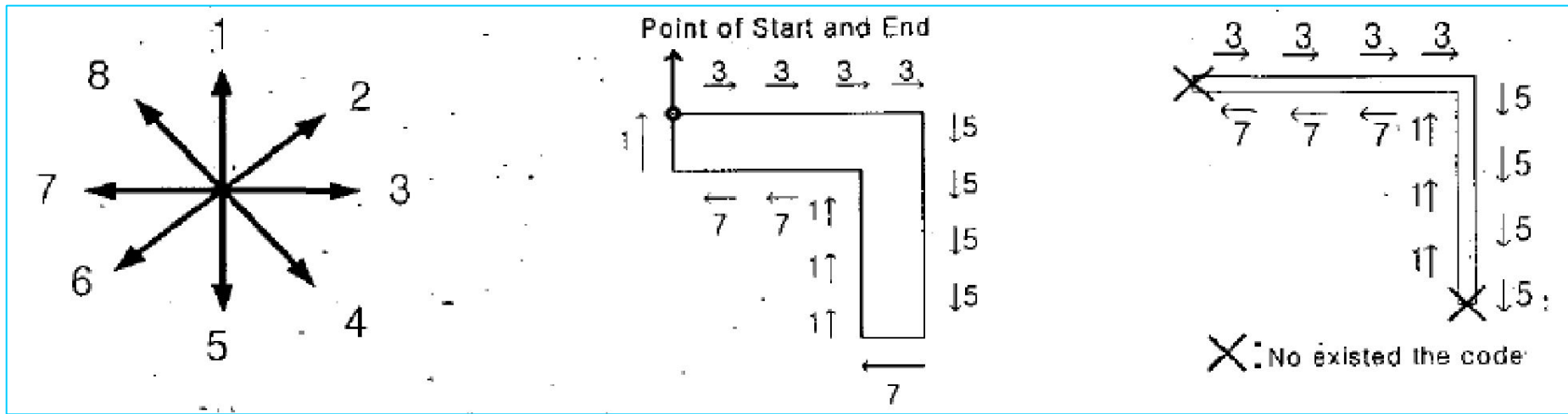


Fig. 6. Formation of closed region by eight-direction code vector : (a) eight-direction code vector , (b) pattern formation of closed region, and (c) no existed code vector

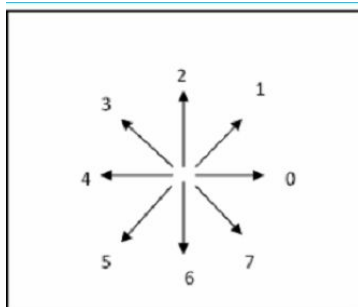


Image Registration

- What is Image Registration?
 - ◆ Aligning images correctly to make systems have better performance.
- Misregistration between images
 - ◆ Translational differences
 - ◆ Scale differences
 - ◆ Rotational differences

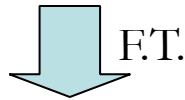
Image Registration : Detecting Translational Parameter

- **Spatial domain approach**
 - Normalized 2D matched filter**
 - ◆ The highest output value is the best translational position.
- **Frequency domain approach**
 - Phase correlation method**

Image Registration : Detecting Translational Parameter

- **Phase correlation method**

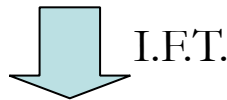
- ◆ $F_2(x, y) = F_1(x-x_0, y-y_0)$



$$F_2(w_x, w_y) = F_1(w_x, w_y) \exp\{-i(w_x x_0 + w_y y_0)\}$$

- ◆ Cross-power spectrum

$$G(w_x, w_y) \equiv \frac{F_1(w_x, w_y) F_2^*(w_x, w_y)}{|F_1(w_x, w_y) F_2(w_x, w_y)|} = \exp\{i(w_x x_0 + w_y y_0)\}$$

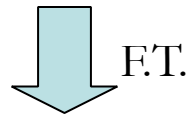


$$G(x, y) = \delta(x - x_0, y - y_0)$$

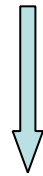
Image Registration : Detecting Scale and Rotational Parameter

- **Detecting rotational parameter**

$$F_2(x, y) = F_1(x \cos \theta_0 + y \sin \theta_0 - x_0, -x \sin \theta_0 + y \cos \theta_0 - y_0)$$



$$F_2(w_x, w_y) = F_1(w_x \cos \theta_0 + w_y \sin \theta_0, -w_x \sin \theta_0 + w_y \cos \theta_0) \exp \{-i(w_x x_0 + w_y y_0)\}$$



❶ Taking magnitudes both sides

❷ Representing in polar form

$$M_2(\rho, \theta) = M_1(\rho, \theta - \theta_0)$$

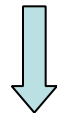
Image Registration : Detecting Scale and Rotational Parameter

- Detecting scale parameter

$$F_2(x, y) = F_1(ax, by)$$



$$F_2(w_x, w_y) = \frac{1}{|ab|} F_1\left(\frac{w_x}{a}, \frac{w_y}{b}\right)$$



Frequency variables to a logarithmic scale

$$F_2(\log w_x, \log w_y) = \frac{1}{|ab|} F_1(\log w_x - \log a, \log w_y - \log b)$$

Bayes Statistical Classifiers

- **Consideration**
 - Randomness of patterns
- **Decision criterion**

Pattern x is labeled as class w_i if

$$\sum_{k=1}^W L_{ki} p(x / w_k) P(w_k) < \sum_{q=1}^W L_{qi} p(x / w_q) P(w_q)$$

L_{ij} : Misclassification loss function

$p(\mathbf{x}/w_i)$: P.d.f. of a particular pattern $\mathbf{x}$ comes from class w_i

$P(w_i)$: Probability of occurrence of class w_i

Bayes Statistical Classifiers

- Decision criterion :
Given L_{ij} is symmetrical function
- Posterior probability decision rule

$$p(x / w_i)P(w_i) > p(x / w_j)P(w_j)$$

$$d_j(x) = p(x / w_j)P(w_j) = P(w_j / x)$$

$d_j(\mathbf{x})$: decision functions

Pattern $\mathbf{x}$ classifies to class j if $d_j(\mathbf{x})$ yields the largest value

Bayes Statistical Classifiers

- **Advantages**

- ◆ Optimization in minimizing the total average loss
in misclassification.

- **Disadvantages**

- ◆ Both $P(w_j)$ and $p(\mathbf{x}/w_j)$ must be known in advance.

Estimation is required.

Performance highly depends on the assumption of the distributions.($P(w_j)$ and $p(\mathbf{x}/w_j)$)

Neural Networks

- **What is Neural Networks?**
 - ◆ Ideas stem from the operation of human neural networks.
 - ◆ Networks of interconnected nonlinear computing elements called neurons.

Neural Networks

- Perceptron : two classes model

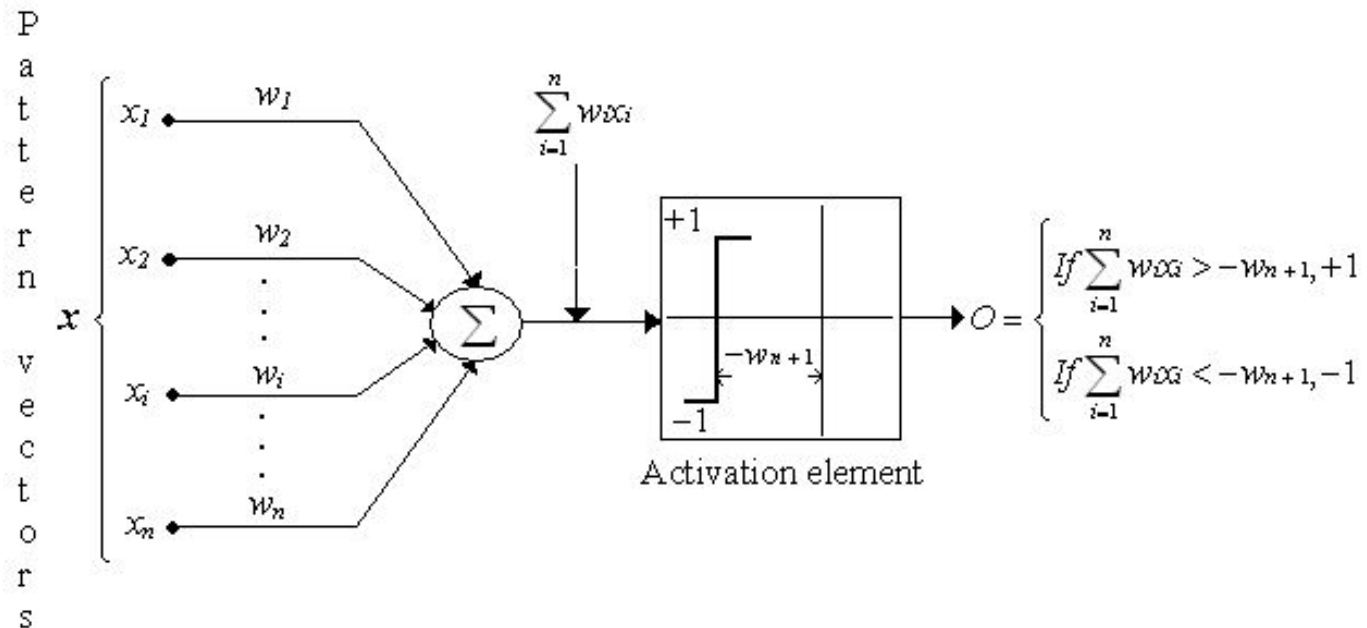


Fig.2 Structure of perceptron

Neural Networks : Multilayer Feedforward Neural Networks

- Basic structure

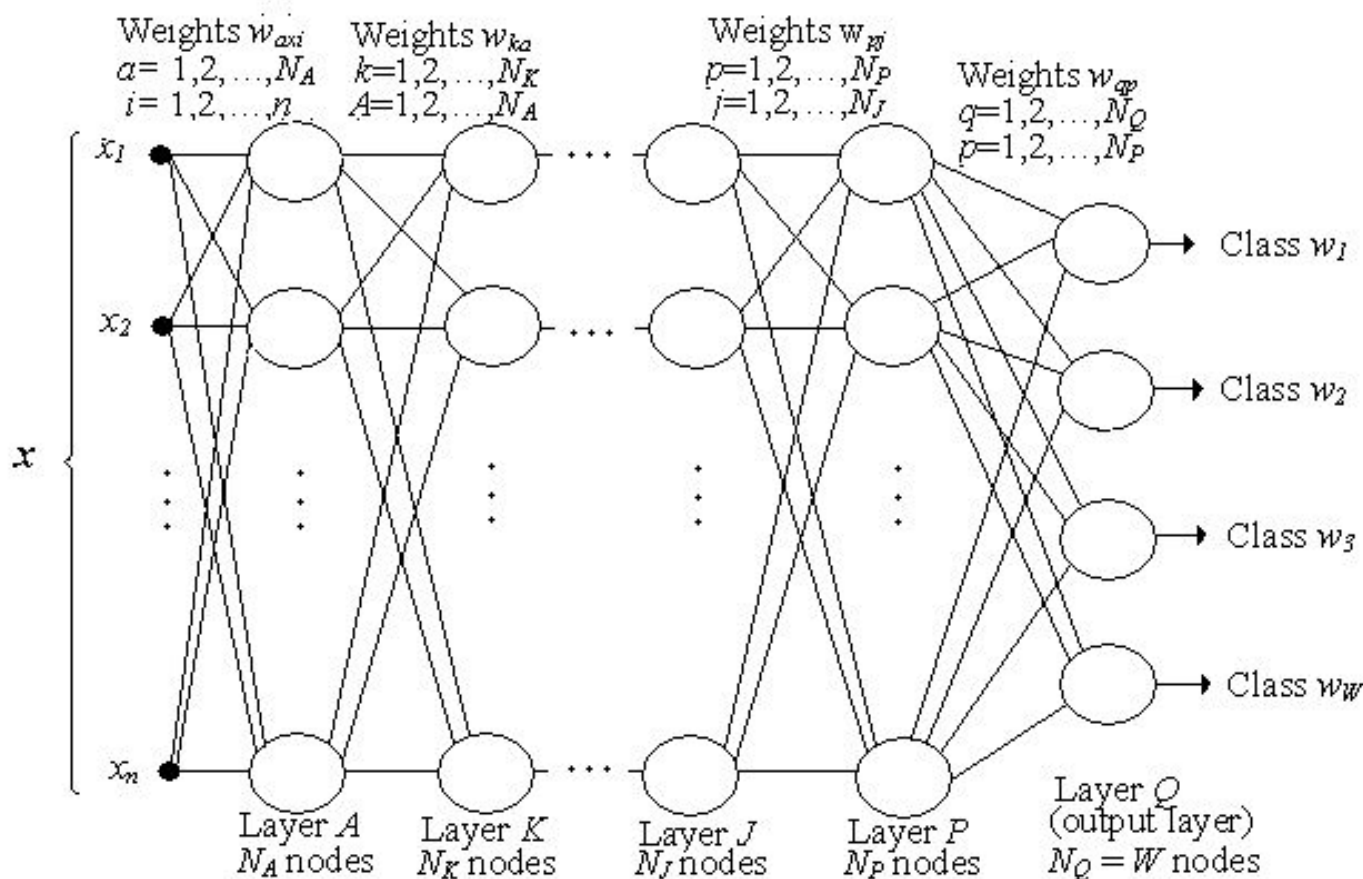


Fig.3 Structure of multilayer feedforward neural networks

Neural Networks : Multilayer Feedforward Neural Networks

- **Training algorithm : back propagation**
 - ◆ Sigmoid activation function

$$h_j(I_j) = \frac{1}{1 + e^{-(\sum_{k=1}^{N_k} w_{jk} O_k + \theta_j) / \theta_0}}$$

$$I_j = \sum_{k=1}^{N_k} w_{jk} O_k$$

$$O_k = h_k(I_k)$$

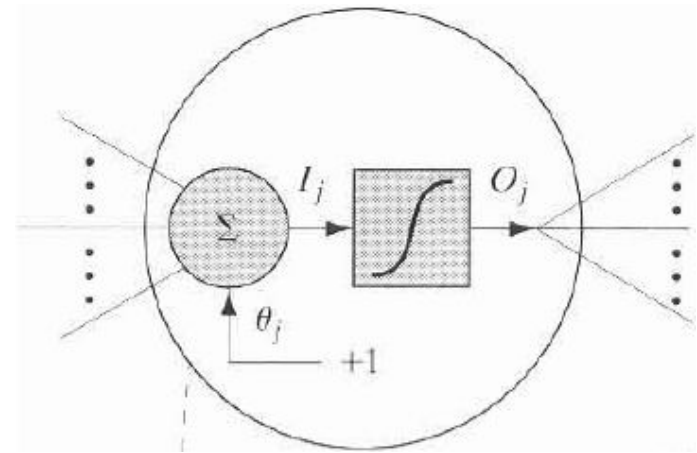


Fig.4 Blowup of a neuron[1]

Neural Networks : Multilayer Feedforward Neural Networks

1. Initialization

Assigning an arbitrary set of weights throughout the network (not equally).

2. Iterative step

a. Computing O_j for each node by using training vector, then generating the error terms for output δ_q , where

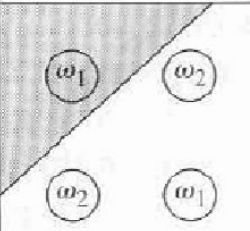
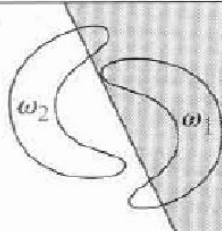
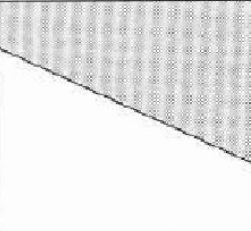
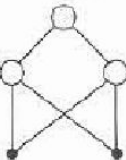
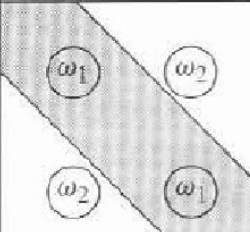
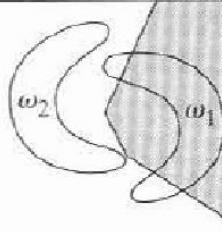
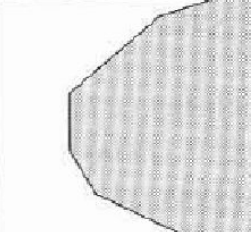
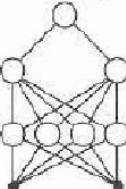
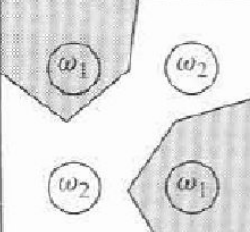
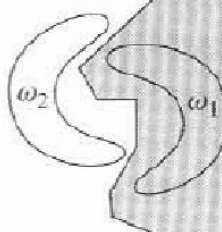
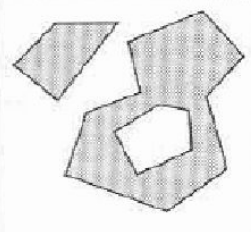
$\delta_q = (r_q - O_q)h'_q(I_q)$, r_q is the desired response.

b. Backward passing appropriate error signal is passed to each node and the corresponding weight changes are made.

Neural Networks

- Decision surface complexity**

Table2 : Decision surface complexity of multilayer feedforward neural networks[1]

Network structure	Type of decision region	Solution to exclusive-OR problem	Classes with meshed regions	Most general decision surface shapes
Single layer 	Single hyperplane			
Two layers 	Open or closed convex regions			
Three layers 	Arbitrary (complexity limited by the number of nodes)			

Syntactic Recognition

- Concerning the structural relation.
- Patterns represent in combinations of primitives.

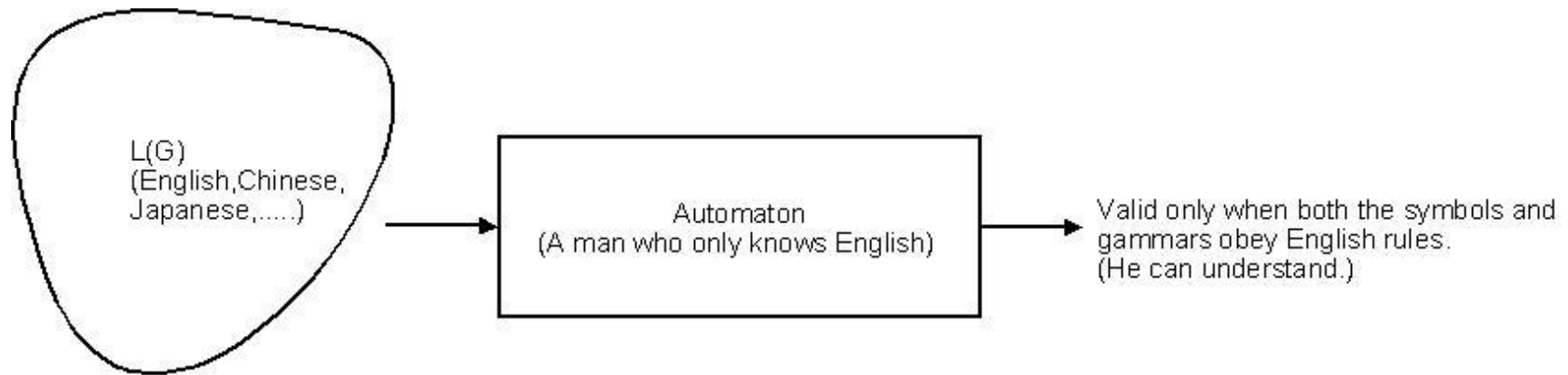


Fig.5 Conceptual diagram of syntactic recognition

Syntactic Recognition : String Case

- Input to the automata are unknown sentences generated by the corresponding grammars respectively.
 - ◆ The grammar $G = (N, \Sigma, P, S)$
 - N is a finite set of variables called *nonterminals*,
 - Σ is a finite set of constants called *terminals*, P is a set of rewriting rules called *productions*, and S in N is called the *starting symbol*.

Syntactic Recognition : String Case

- An example

$N=\{A,B,S\}, \Sigma=\{a,b,c\}$

$P=\{S \rightarrow aA, A \rightarrow bA, A \rightarrow bB, B \rightarrow c\}$

$S \rightarrow aA \rightarrow abA \rightarrow abbA \rightarrow \dots \rightarrow abbbbbc$

$L(G)=\{ab^n c \mid n \geq 1\}$

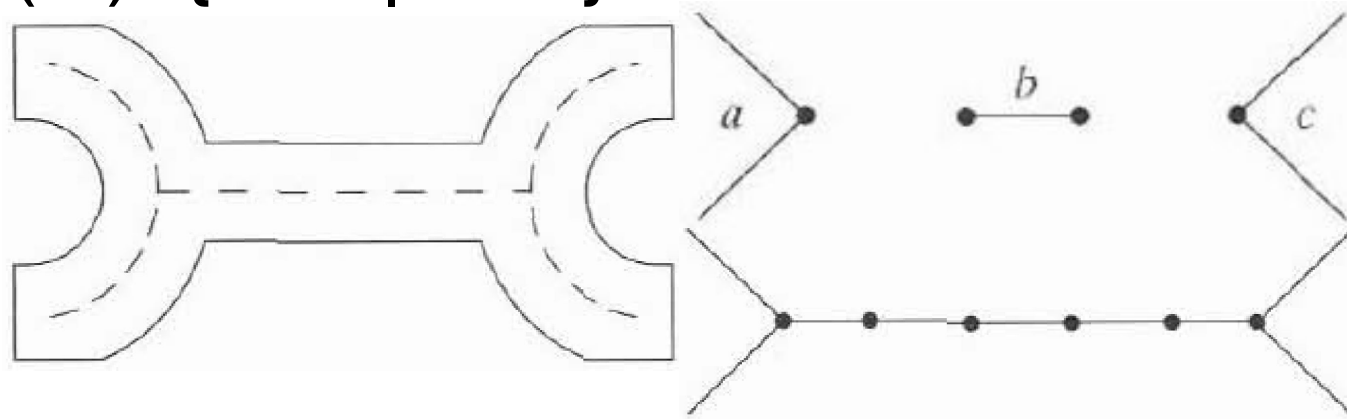


Fig.6 An example of string language[1]

Syntactic Recognition : String Case

- ◆ The finite automata $A_f = (Q, \Sigma, \delta, q_0, F)$
 - Q is a finite, nonempty set of *states*,
 - Σ is a finite input *alphabet*,
 - δ is a *mapping* from $Q \times \Sigma$ into the collection of all subsets of Q ,
 - q_0 is the *starting state*, and
 - F is a set of *final states*.

Syntactic Recognition : String Case

- A simple automaton

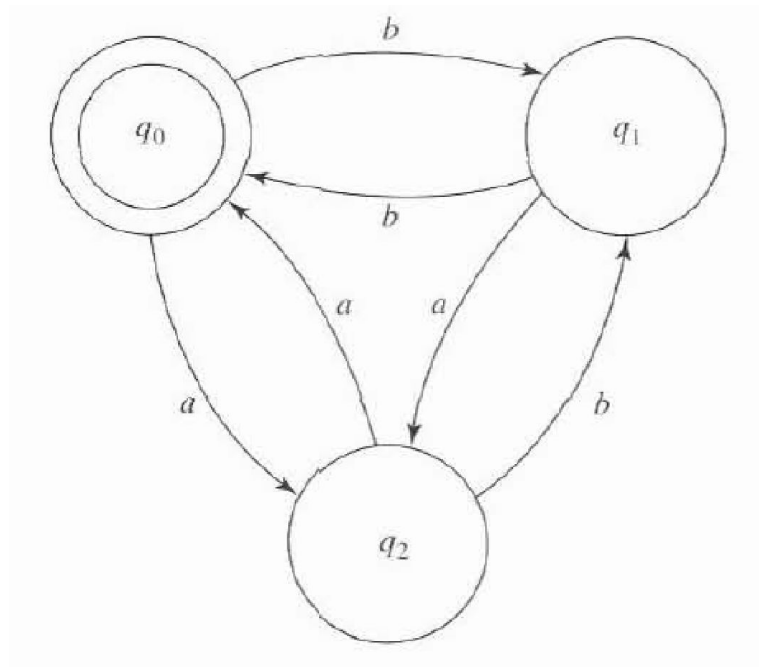


Fig.7 State machine of the automaton[1]

Invalid input string : bababbb

Valid input string : aaabbbb

$$A_f = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$F = \{q_0\}$$

$$\delta(q_0, a) = \{q_2\}$$

$$\delta(q_0, b) = \{q_1\}$$

$$\delta(q_1, a) = \{q_2\}$$

$$\delta(q_1, b) = \{q_0\}$$

$$\delta(q_2, a) = \{q_0\}$$

$$\delta(q_2, b) = \{q_1\}$$

Syntactic Recognition : String Case

- Conversion between regular grammar and corresponding automaton states.

$$G = (N, \Sigma, P, S)$$

$$X_0 \equiv S$$

$$N = \{X_0 \sim X_n\}$$



$$Af = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, \dots, q_n, q_{n+1}\}$$

The mappings in δ are obtained by using the following two rules, for a in Σ , and each i and j , with $0 \leq i \leq n$, $0 \leq j \leq n$,

1. If $X_i \rightarrow aX_j$ is in P , then $\delta(q_i, a)$ contains q_j .
2. If $X_i \rightarrow a$ is in P , then $\delta(q_i, a)$ contains q_{n+1} .

Syntactic Recognition : String Case

- Grammars are not known in advance, we need to learn the automata from sample patterns.
- An unknown grammar G and a finite sets of samples R^+

$$h(z, R^+, k) = \{w \mid zw \text{ in } R^+, |w| \leq k\}, z \text{ belongs to } \Sigma^*$$

$$Q = \{q \mid q = h(z, R^+, k) \text{ for } z \text{ in } \Sigma^*\}$$

$$\delta(q, a) = \{q' \text{ in } Q \mid q' = h(za, R^+, k), \text{ with } q = h(z, R^+, k)\}$$

$$q_0 = h(\lambda, R^+, k)$$

$$F = \{q \mid q \text{ in } Q, \lambda \text{ in } q\}$$

Syntactic Recognition : String Case

- An example of learning automaton structure from a given sample set $R^+ = \{a, ab, abb\}$. ($k=1$)

❶ Determining $h(z, R^+, k)$

$$z = \lambda \quad h(\lambda, R^+, 1) = \{w \mid \lambda w \text{ in } R^+, |w| \leq 1\}$$

$$= \{a\}$$

λ is a empty string set

$$= q_0$$

$$z = a \quad h(a, R^+, 1) = \{w \mid aw \text{ in } R^+, |w| \leq 1\}$$

$$= \{\lambda, b\}$$

$$= q_1$$

$$z = ab \quad h(ab, R^+, 1) = \{w \mid abw \text{ in } R^+, |w| \leq 1\}$$

$$= \{\lambda, b\}$$

$$= q_1$$

$$z = abb \quad h(abb, R^+, 1) = \{w \mid abbw \text{ in } R^+, |w| \leq 1\}$$

$$= \{\lambda\}$$

$$= q_2$$

Syntactic Recognition : String Case

② Obtaining mapping function

$Q = \{q_0, q_1, q_2, q_3\}$, q_3 denotes empty set state

$$h(\lambda, R^+, 1) = q_0, \quad z = \lambda$$

$$\delta(q_0, a) = h(\lambda a, R^+, 1) = h(a, R^+, 1) = q_1$$

$$\delta(q_0, b) = h(\lambda b, R^+, 1) = h(b, R^+, 1) = q_3$$

$$h(a, R^+, 1) = h(ab, R^+, 1) = q_1$$

$$\delta(q_1, a) = h(aa, R^+, 1) = h(aba, R^+, 1) = q_3$$

$$\delta(q_1, b) \supseteq h(ab, R^+, 1) = q_1 \quad \delta(q_1, b) \supseteq h(abb, R^+, 1) = q_2$$

$$\delta(q_1, b) = \{q_1, q_2\}$$

$$\delta(q_2, a) = \delta(q_2, b) = \delta(q_3, a) = \delta(q_3, b) = q_3$$

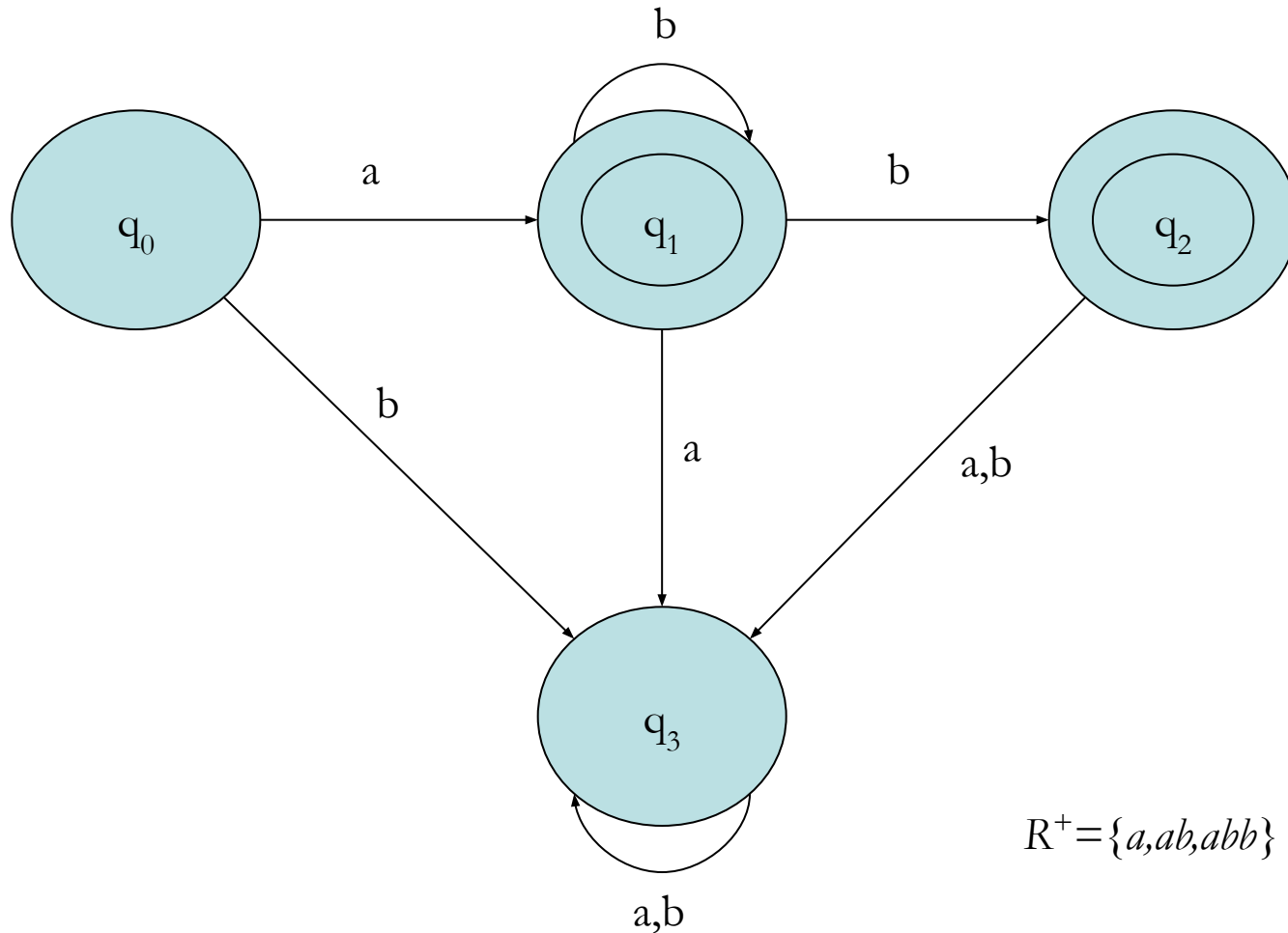
③ Obtaining final state F

$$q_1 = \{\lambda, b\} \quad q_2 = \{\lambda\}$$

$$F = \{q_1, q_2\}$$

Syntactic Recognition : String Case

- State diagram for the finite automaton inferred from the sample set R^+



Syntactic Recognition : String Case

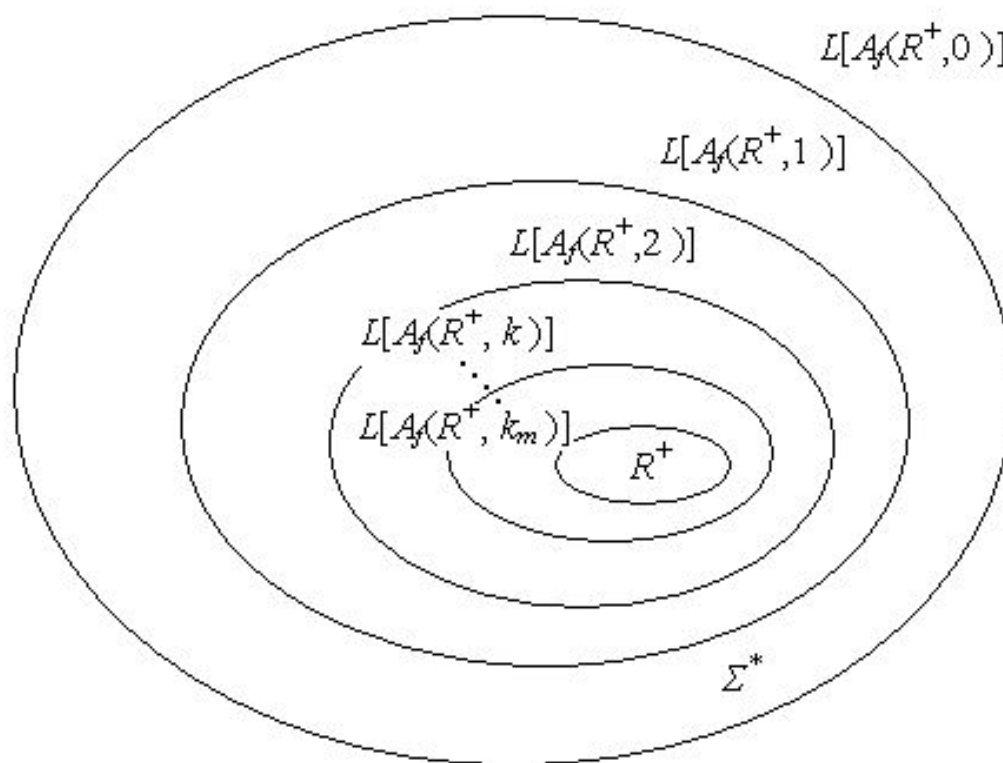


Fig.8 Graphic relation between k and $L[A_f(R^+, k + 1)]$

Face Recognition

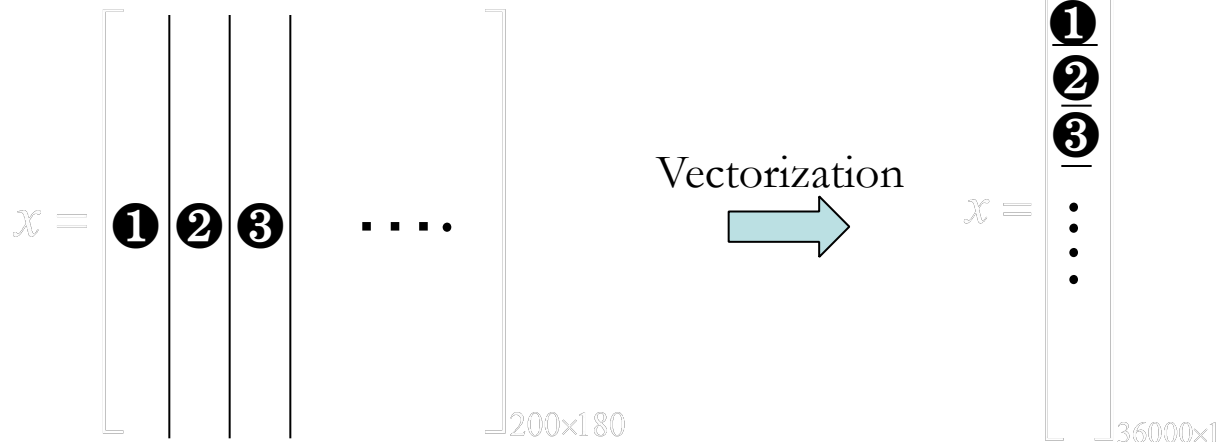
- User-friendly pattern recognition application
- Weakness of face recognition
 - Illumination problems
 - Pose problems(profile or frontal view)



Fig.9 Examples of illumination problems[9]

Face Recognition : Eigenspace-Based Approach

- Eigenspace-based approach
 - ◆ A holistic approach
 - ◆ Reducing the high dimensionality problem , and large computational complexity.



A face image of size 200×180

Face Recognition : Standard Eigenspace-Based Approach

- Standard Eigenspace-based approach
 - ◆ Given a set of training face images, computing the eigenvectors of the distribution of face images within the entire image space .(PCA method)

$$\Psi = \frac{1}{M} \sum_{n=1}^M \Gamma_n$$

Size of $N^2 \times N^2$

Γ_n : face vectors

Size of $N^2 \times M$

$$C = \frac{1}{M} \sum_{n=1}^M \Phi_n \Phi_n^T = A A^T$$

$$A = [\Phi_1, \Phi_2, \dots, \Phi_M]$$

C : Covariance matrix of training set

M : Number of training face images

$$\Phi_i = \Gamma_i - \Psi$$

Ψ : Mean vector

Face Recognition : Standard Eigenspace-Based Approach

- Standard Eigenspace-based approach
 - ◆ Given a set of training face images, computing the eigenvectors of the distribution of face images within the entire image space. (PCA method)

$$\Psi = \frac{1}{M} \sum_{n=1}^M \Gamma_n$$

length N^2

mean face vector

$$\Phi_i = \Gamma_i - \Psi$$

length N^2

mean-centred vector, $i = 1..M$

$$A = [\Phi_1, \Phi_2, \dots, \Phi_M]$$

$N^2 \times M$

data matrix

$$C = \frac{1}{M} \sum_{n=1}^M \Phi_n \Phi_n^T = AA^T$$

$N^2 \times N^2$

covariance matrix

Γ_n : face vectors

Φ_i : mean-centred face vector

C : covariance matrix of training set

Ψ : mean vector

A : matrix of training vectors

M : number of training face images

Reducing the Eigenvalue Problem

The covariance matrix C is too big. We can reduce the eigenvalue problem from order $N^2 \times N^2$ down to $M \times M$ using the following analysis.

THE DERIVATION

Start from the small $M \times M$ problem

$$A^T A v_i = \mu_i v_i$$

premultiply both sides by A



gives the large $N^2 \times N^2$ eigenvectors

$$\begin{aligned} C(A v_i) &= A A^T (A v_i) = A(A^T A) v_i \\ &= A(\mu_i v_i) = \mu_i (A v_i) \end{aligned}$$

So each $A v_i$ is an eigenvector of C , sharing the same eigenvalue μ_i

Solve the cheap $M \times M$ problem, then map the results up.

NOTATION

A	M mean-centred images
$C = AA^T$	covariance, $N^2 \times N^2$
$A A^T$	small matrix, $M \times M$
v_i	eigenvectors of $A^T A$
μ_i	eigenvalues of $A^T A$ and C
$A v_i$	eigenvectors of C

WHY IT MATTERS

N^2 = pixels per image
(often tens of thousands)

M = number of training images
(often only a few hundred)

C has N^2 eigenvectors, but at most M of them are useful – exactly the ones this trick recovers.

Face Recognition : Standard Eigenspace-Based Approach

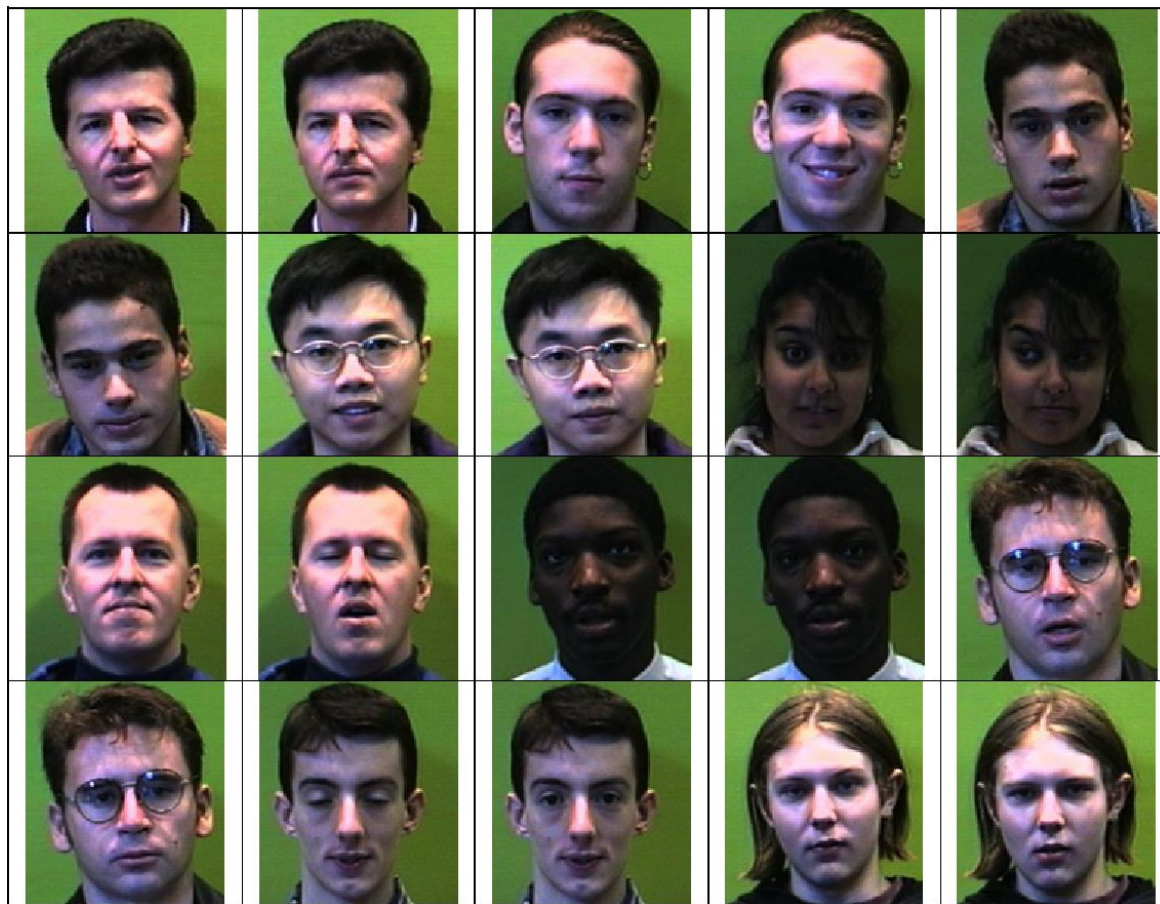


Fig.10 Training set



Fig.11 Mean face

Face Recognition : FLD

Eigenspace-Based Approach

- Mathematical Expression

- ① Selecting projection unitary vector $\mathbf{u}$ s.t. $\Upsilon(\mathbf{u})$ to be maximized

$$\gamma(\mathbf{u}) = \frac{\mathbf{u}^T \mathbf{S}_b \mathbf{u}}{\mathbf{u}^T \mathbf{S}_w \mathbf{u}}$$

$$\mathbf{S}_b = \sum_{i=1}^{NC} P(C_i) (\mathbf{m}(i) - \mathbf{m})(\mathbf{m}(i) - \mathbf{m})^T$$

$$\mathbf{S}_w = \sum_{i=1}^{NC} P(C_i) E[(\mathbf{x}(i) - \mathbf{m}(i))(\mathbf{x}(i) - \mathbf{m}(i))^T]$$

$\mathbf{S}_b$: Measuring the separation between the individual class means respect to the global mean face

$\mathbf{S}_w$: Measuring the separation between vectors of each class respect to their own class mean

- ② Using Lagrange multiplier and set $\mathbf{u}^T \mathbf{S}_w \mathbf{u} = 1$ be the constraint condition

$$\mathbf{S}_b \mathbf{w}^k = \lambda_k \mathbf{S}_w \mathbf{w}^k \quad : \text{Generalized eigenvalue problem}$$

Thank You